

AFFAN STORE - API SPECIFICATION

Panduan Lengkap Integrasi Front End • NestJS + Prisma ORM + JWT Auth

1. Konfigurasi Lingkungan & Parameter Dasar

BASE URL (LOKAL):	<code>http://localhost:3000</code>
SWAGGER DOCS:	<code>http://localhost:3000/api</code>
STATIC ASSETS (IMG):	<code>http://localhost:3000/uploads/<filename></code>
HEADER AUTH:	<code>Authorization: Bearer <access_token></code>

STATUS CORS: `app.enableCors()` sudah aktif. Request dari localhost frontend tidak akan diblokir browser.

2. Master Data & Enums Acuan

NAMA ENUM	NILAI YANG VALID	KETERANGAN / FUNGSIONALITAS
Role	<code>USER, ADMIN</code>	Hak akses akun. Default saat registrasi adalah USER
PaymentMethod	<code>CASH, QRIS</code>	Metode pembayaran pesanan checkout
ShippingType	<code>COD, NON_COD</code>	Tipe pengiriman barang pesanan
OrderStatus	<code>PENDING, PAID, CANCELLED</code>	Status siklus perjalanan transaksi

3. Aturan Bisnis Transaksi (Business Logic)

- Sinkronisasi Metode Bayar & Pengiriman (Validasi Strict Backend):
 - Jika `paymentMethod = "CASH"`, maka `shippingType` WAJIB bernilai `"COD"`.
 - Jika `paymentMethod = "QRIS"`, maka `shippingType` WAJIB bernilai `"NON_COD"`.
 - Kombinasi yang tidak cocok akan ditolak otomatis dengan response 400 Bad Request.
- Atomic Database Transaction (Pengurangan Stok Otomatis):
 - Stok produk langsung dipotong saat order terbuat secara atomic dalam `$transaction`.
- Restock Otomatis pada Pembatalan:
 - Jika order `PENDING` dibatalkan (oleh User atau Admin), stok dikembalikan otomatis.

4. Modul Autentikasi Pengguna (/auth)

POST /auth/register

PUBLIC

Mendaftarkan customer baru ke dalam sistem. Role otomatis diset ke USER.

REQUEST PAYLOAD / BODY:

```
{
  "name": "Budi Santoso",
  "email": "budi@example.com",
  "password": "password123" // min 6 karakter
}
```

RESPONSE CONTOH (200 / 201):

```
{
  "id": 1,
  "name": "Budi Santoso",
  "email": "budi@example.com",
  "role": "USER",
  "createdAt": "2026-10-06T06:00:00.000Z"
}
```

POST /auth/login

PUBLIC

Login akun untuk mendapatkan Bearer JWT Access Token.

REQUEST PAYLOAD / BODY:

```
{
  "email": "budi@example.com",
  "password": "password123"
}
```

RESPONSE CONTOH (200 / 201):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": 1, "name": "Budi Santoso", "email": "budi@example.com", "role": "USER" }
}
```

5. Modul Katalog Produk (/products)

GET /products

PUBLIC

Mengambil seluruh daftar produk aktif toko, diurutkan dari yang terbaru (createdAt: desc).

RESPONSE CONTOH (200 / 201):

```
[
  {
    "id": 1,
    "name": "Sepatu Running Adidas",
    "price": 650000,
    "category": "Sepatu",
    "stock": 15,
    "imageUrl": "/uploads/product-171800-123.jpg",
    "isActive": true
  }
]
```

GET /products/:id

PUBLIC

Melihat detail lengkap 1 produk spesifik berdasarkan parameter ID produk.

RESPONSE CONTOH (200 / 201):

```
{
  "id": 1,
  "name": "Sepatu Running Adidas",
  "description": "Bahan empuk",
  "price": 650000,
  "category": "Sepatu",
  "stock": 15,
  "imageUrl": "/uploads/product-171800-123.jpg"
}
```

5. Modul Produk (Lanjutan - Khusus Admin)

POST /products

ADMIN ONLY

Tambah produk baru. Request WAJIB multipart/form-data jika mengunggah file gambar (field: image).

REQUEST PAYLOAD / BODY:

Form-Data Fields:

- name (text, wajib), price (number, wajib), category (text, wajib), stock (number, wajib)
- description (text, opsional), image (file binary, opsional: JPG/PNG)

RESPONSE CONTOH (200 / 201):

```
{
  "id": 2,
  "name": "Tas Ransel",
  "price": 250000,
  "category": "Tas",
  "stock": 20,
  "imageUrl": "/uploads/product-171999-456.png"
}
```

PATCH /products/:id

ADMIN ONLY

Update produk / ganti foto produk (multipart/form-data. Kirim properti yang ingin diubah saja).

DELETE /products/:id

ADMIN ONLY

Menghapus data produk dari katalog berdasarkan ID produk.

6. Modul Pemesanan & Transaksi (/orders)

POST /orders

[USER LOGIN](#)

Checkout pesanan baru. Mengurangi stok produk secara atomic di database.

REQUEST PAYLOAD / BODY:

```
{
  "productId": 1,
  "quantity": 2,
  "paymentMethod": "QRIS", // Pilihan: "CASH" atau "QRIS"
  "shippingType": "NON_COD" // Wajib "NON_COD" jika QRIS; Wajib "COD" jika CASH
}
```

RESPONSE CONTOH (200 / 201):

```
{
  "id": 10,
  "userId": 1,
  "totalAmount": 1300000,
  "paymentMethod": "QRIS",
  "shippingType": "NON_COD",
  "status": "PENDING",
  "orderItems": [
    { "productId": 1, "quantity": 2, "price": 650000, "product": { "name": "Sepatu Running" } }
  ]
}
```

GET /orders

[USER LOGIN](#)

Menampilkan riwayat pesanan milik pengguna yang sedang terautentikasi (berserta detail produk).

RESPONSE CONTOH (200 / 201):

```
[
  {
    "id": 10,
    "totalAmount": 1300000,
    "status": "PENDING",
    "orderItems": [{ "product": { "name": "Sepatu Running Adidas" } }]
  }
]
```

GET /orders/:id

[USER / ADMIN](#)

Melihat detail 1 pesanan berdasarkan ID. Dibatasi hanya untuk pemilik pesanan atau akun Admin.

PATCH /orders/:id/cancel

[PEMILIK ORDER](#)

Membatalkan pesanan sendiri jika status masih PENDING. Stok produk langsung otomatis dikembalikan.

RESPONSE CONTOH (200 / 201):

```
{
  "id": 10,
  "status": "CANCELLED",
  "updatedAt": "2026-10-06T06:20:00.000Z"
}
```

GET /orders/admin/all

[ADMIN ONLY](#)

Melihat seluruh pesanan dari semua pengguna (lengkap dengan data akun pemesan & barang yang dibeli).

PATCH /orders/:id/status

[ADMIN ONLY](#)

Mengubah status pesanan oleh admin (contoh: verifikasi PAID atau pembatalan CANCELLED).

REQUEST PAYLOAD / BODY:

```
{
  "status": "PAID" // Nilai valid: "PENDING" | "PAID" | "CANCELLED"
}
```

RESPONSE CONTOH (200 / 201):

```
{
  "id": 10,
  "status": "PAID",
  "updatedAt": "2026-10-06T06:25:00.000Z"
}
```

7. Modul Manajemen Pengguna (/users)

METHOD & ROUTE		HAK AKSES	FUNGSI & PENJELASAN
GET	/users	ADMIN ONLY	Melihat seluruh daftar user terdaftar
POST	/users	ADMIN ONLY	Admin membuat user baru secara manual
GET	/users/:id	USER / ADMIN	Melihat detail data profil spesifik
PATCH	/users/:id	USER / ADMIN	Update profil (nama, email, password)
DELETE	/users/:id	ADMIN ONLY	Menghapus akun pengguna dari database

8. Rekomendasi Setup Front End (Axios Client)

```
// src/services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3000',
});

// Otomatis pasang Bearer Token jika user sudah login
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor respon jika token kadaluwarsa (401 Unauthorized)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```